

Performance Engineering (2360013) — Assignment 1

Rami Habib, ID 325420180. Instructor: Nadav Amit.

VM: course-08, a KVM guest with an Intel Xeon Gold 5420+, 4 vCPUs, kernel 7.0.0-15, ext4.

Short version of my findings:

- **Task 1:** the pipe gets faster under CPU load because the scheduler co-locates the two pipe processes on one core. With idle CPUs around it spreads them across cores, and every round-trip then pays a cross-core wakeup; load removes the idle CPUs so they end up together.
- **Task 2:** v2 reads faster because its 4 MiB writes make the kernel store the file in 2 MiB folios, which get mapped as huge pages. That cuts TLB misses by about 475x on a workload that is otherwise TLB-bound.

The raw command outputs I relied on are in the appendix.

Task 1: Pipe latency under background load

The phenomenon

`lat_pipe` measures the round-trip latency of a Unix pipe. On its own it reports about 11 μ s. When I run `stress-ng --cpu 3` in the background, it drops to about 5 μ s, so adding CPU load makes it roughly 2x faster, which is the opposite of what I expected.

Baseline, 10 runs each, median:

	latency
no load	11.1 μ s
with load (stress-ng --cpu 3)	5.3 μ s

The effect is much bigger than the run-to-run spread, so it is real.

What could cause it

The benchmark does a fixed amount of work per round-trip, so the latency is basically (work / CPU speed) + (cost of waking up the other process). Background load could help in three ways: the CPU could run at a higher clock under load (frequency), an idle CPU could be paying a sleep/wakeup cost (idle states), or the two pipe processes could be placed differently by the scheduler (placement).

This is a KVM guest with no `cpufreq` or `cpuidle` drivers exposed (I checked with `cpupower`), so I could not toggle frequency or C-states directly. I probed indirectly with `perf` instead.

Ruling out frequency

I compared `cycles` to `ref-cycles` in `perf stat`. `ref-cycles` count at the fixed nominal clock, so their ratio is the real frequency. It was the same (1.35) with and without load, which means that when the CPU is actually running it runs at the same speed either way. Frequency is not it.

Separating idle from placement

Pinning both pipe processes to a single core (no load) gave 5.4 μ s, as fast as the loaded case. But that changes two things at once (the core can no longer go idle, and both ends are now on the same core), so it does not tell me which one matters.

To separate them I pinned both ends to two cores and ran it twice: once letting the cores idle, once with a busy spinner on each core so they never idle.

	latency
same core	5.4 μ s
two cores, idle	11.2 μ s
two cores, kept busy	10.9 μ s

Keeping the cores busy did not help, so idle/halt cost is not the cause. The thing that matters is same-core versus cross-core.

The cause: scheduler placement

When there are idle CPUs, the scheduler spreads the two pipe processes onto different cores, so every round-trip is a cross-core wakeup (an inter-processor interrupt, plus the pipe buffer and task data bouncing between the two cores' caches). Background load keeps all the CPUs busy, so the scheduler puts both processes on the same core, where a context switch is about twice as cheap.

Two pieces of evidence back this up. A perf sched callgraph of the no-load run shows the two pipe processes on different cores (cpu0 and cpu3), each handing the CPU to the idle task when it blocks. And an off-CPU latency histogram shows the per-handoff wait dropping from the 8-16 μ s bucket (no load) to the 2-4 μ s bucket (with load), which matches the latency change.

Kernel source

The placement decision is in `kernel/sched/fair.c`. On a wakeup, `select_task_rq_fair` takes the fast path to `select_idle_sibling`, and inside it (line 110) `select_idle_cpu` scans the cache domain for an idle CPU and returns it. So with idle CPUs available the woken pipe process is sent to a free core, away from its partner. Under load there is no idle CPU to find, so it stays on the waker's core and the two processes run together. That co-location is why load makes the pipe faster.

Task 2: Two prepares, one run

The phenomenon

I create the same 64 MiB file twice: v1 with the default (~16 KiB) write block size, and v2 with 4 MiB writes. Then I run the same random 4 KiB mmap read test against each. v2 comes out about 9% faster:

	reads/s
v1	1,942,670
v2	2,124,442 (+9.4%)

Both files hold the same data and the read command is identical, so the only difference is how each file was written during prepare.

Ruling out disk fragmentation

The obvious guess is that v1's small writes fragment the file on disk. But the file is only 64 MiB and the VM has plenty of RAM, so after prepare the whole file sits in the page cache. perf stat confirms this: during the run the page faults are all minor (page-faults equals minor-faults, zero major faults), which means nothing is read from disk. If the disk is never touched, its layout cannot affect the result. filefrag also shows both files are basically contiguous (2 extents vs 1). So fragmentation is not the cause.

What actually differs: folio size

The real difference is how the kernel stores each file in the page cache. I traced the folio sizes built during prepare with bpftrace (on `mm_filemap_add_to_page_cache`):

	folios in the page cache
v1 (16 KiB writes)	16 KiB folios
v2 (4 MiB writes)	2 MiB folios (the whole file)

Bigger writes let the kernel build bigger folios.

Why bigger folios read faster

v2's 2 MiB folios get mapped as huge pages. `/proc/<pid>/smaps_rollup` during a run shows:

	FilePmdMapped
v1	0
v2	65,536 kB (the whole 64 MiB file)

This matters because the benchmark is limited by the TLB. With 4 KiB pages the TLB only covers a few MiB, so random reads across a 64 MiB file miss almost every time. perf stat shows exactly that:

	dTLB-load-misses
v1	8,790,240
v2	18,488 (about 475x fewer)

A handful of 2 MiB pages cover the whole file, so v2 almost never misses the TLB. The LLC cache-misses are about the same for both (130M vs 127M), so this is a TLB effect and not a cache effect. Fewer TLB misses means fewer page-table walks, and that is the ~9% speedup.

Kernel source

- `mm/readahead.c`, `page_cache_ra_order`: the folio order is capped by the I/O size (`new_order = min(..., ilog2(ra->size))`). v2's 4 MiB writes allow a 2 MiB folio; v1's small writes do not.
- `mm/memory.c`, `do_set_pmd`: installs a 2 MiB huge-page mapping, but only when the folio is already PMD-sized. It fires for v2 and falls back to 4 KiB pages for v1.

A note on reproducing it: the effect is fragile

The gap only appears on a freshly-booted VM, and it fades as the machine runs. When it is present, v2's folios are 2 MiB and get PMD-mapped (`FilePmdMapped = 64 MiB`, `do_set_pmd` fires). When it has faded, `do_set_pmd` is **never even called**. That is the key clue: `finish_fault` only calls `do_set_pmd` when `folio_test_pmd_mappable(folio)` is true, i.e. when the folio is at least PMD-sized (2 MiB). So in the faded state the folios are no longer 2 MiB at fault time, which is why the huge mapping is never attempted and the gap disappears.

The trigger is therefore the system's **memory state**, which controls what order of folio v2's write can build (and keep), not a kernel setting. I checked the call site directly: `do_set_pmd` is gated by `folio_test_pmd_mappable`, not by `CONFIG_READ_ONLY_THP_FOR_FS` (which I had wrongly blamed in an earlier draft). On a long-running, fragmented system v2's folios end up sub-2 MiB and the gap vanishes; a reboot restores clean memory, the 4 MiB write produces 2 MiB folios again, and the gap returns. I did not fully isolate which aspect of the aged memory state pushes the folio below 2 MiB, so I report what is verified: the gap tracks whether v2's folio is PMD-sized at fault time.

Appendix: raw command outputs

Task 1

logs/task1_20260610_1427.log

baseline: lat_pipe x10, no-load then with-load

```
Script started on 2026-06-10 14:27:48+00:00 [COMMAND="bash /tmp/task1_p01.sh" <not executed on terminal>]
##### PHASE 0 - environment as shipped #####
wrote task1/env_asshipped.txt
##### PHASE 1 - baseline no-load (n=10) #####
# Task1 Phase 1 - lat_pipe, NO load, n=10 - 2026-06-10T14:27:48+00:00
Pipe latency: 11.2042 microseconds
Pipe latency: 10.4985 microseconds
Pipe latency: 11.1315 microseconds
Pipe latency: 11.1770 microseconds
Pipe latency: 11.1627 microseconds
Pipe latency: 11.1287 microseconds
Pipe latency: 11.1157 microseconds
Pipe latency: 11.0622 microseconds
Pipe latency: 10.5657 microseconds
Pipe latency: 9.8751 microseconds
##### PHASE 1 - baseline with load: stress-ng --cpu 3 (n=10) #####
# Task1 Phase 1 - lat_pipe, WITH load (stress-ng --cpu 3), n=10 - 2026-06-10T14:28:04+00:00
Pipe latency: 5.2355 microseconds
Pipe latency: 5.6613 microseconds
Pipe latency: 5.2755 microseconds
Pipe latency: 6.3949 microseconds
Pipe latency: 5.5291 microseconds
Pipe latency: 5.5314 microseconds
Pipe latency: 4.7368 microseconds
Pipe latency: 4.9441 microseconds
Pipe latency: 5.0993 microseconds
Pipe latency: 4.8185 microseconds
##### DONE #####

Script done on 2026-06-10 14:30:34+00:00 [COMMAND_EXIT_CODE="0"]
```

task1/p2_perfstat_noload.txt

perf stat, no load (frequency check)

Pipe latency: 69.6173 microseconds

Performance counter stats for '/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe':

1513.48 msec	task-clock
1619968404	cycles
667048034	instructions
1202635280	ref-cycles
29252	context-switches
2	cpu-migrations

1.571171194 seconds time elapsed

0.587697000 seconds user

0.891950000 seconds sys

task1/p2_perfstat_load.txt

perf stat, with load

Pipe latency: 69.1056 microseconds

Performance counter stats for '/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe':

28749.27 msec	task-clock
73484767995	cycles
26922915348	instructions
54567480720	ref-cycles
33982	context-switches
6	cpu-migrations

29.746661716 seconds time elapsed

27.719841000 seconds user

1.001958000 seconds sys

task1/p2_pin1core_noload.txt

both pipe ends pinned to one core

Pipe latency: 5.4472 microseconds
Pipe latency: 5.3650 microseconds
Pipe latency: 5.4265 microseconds
Pipe latency: 5.4146 microseconds
Pipe latency: 5.4262 microseconds

task1/p2b_pin2core_idle.txt

two cores, allowed to idle

Pipe latency: 11.2613 microseconds
Pipe latency: 11.3220 microseconds
Pipe latency: 11.2336 microseconds
Pipe latency: 11.2479 microseconds
Pipe latency: 11.0964 microseconds

task1/p2b_pin2core_busy.txt

two cores, kept busy

```
Pipe latency: 10.3362 microseconds
Pipe latency: 10.9540 microseconds
Pipe latency: 11.0637 microseconds
Pipe latency: 11.0859 microseconds
Pipe latency: 11.1015 microseconds
```

task1/p4_offcpu_hist_noload.txt

off-CPU latency histogram, no load

```
stdin:2:125-151: WARNING: Return value discarded.
tracepoint:sched:sched_switch /args->next_comm=="lat_pipe"/ { $s=@t[args->next_pid]; if($s){
@off_us=hist((nsecs-$s)/1000); delete(@t[args->next_pid]); } }
```

~~~~~

Attached 2 probes

Pipe latency: 17.5059 microseconds

@off\_us:

|              |        |  |  |
|--------------|--------|--|--|
| [2, 4)       | 9993   |  |  |
| [4, 8)       | 353    |  |  |
| [8, 16)      | 139094 |  |  |
| [16, 32)     | 33289  |  |  |
| [32, 64)     | 944    |  |  |
| [64, 128)    | 157    |  |  |
| [128, 256)   | 36     |  |  |
| [256, 512)   | 16     |  |  |
| [512, 1K)    | 17     |  |  |
| [1K, 2K)     | 5      |  |  |
| [2K, 4K)     | 2      |  |  |
| [4K, 8K)     | 0      |  |  |
| [8K, 16K)    | 0      |  |  |
| [16K, 32K)   | 0      |  |  |
| [32K, 64K)   | 0      |  |  |
| [64K, 128K)  | 0      |  |  |
| [128K, 256K) | 0      |  |  |
| [256K, 512K) | 0      |  |  |
| [512K, 1M)   | 2      |  |  |

@t[77549]: 1643768205293775

@t[77548]: 1643768206232093

@t[77535]: 1643768206467861

## **task1/p4\_offcpu\_hist\_load.txt**

*off-CPU latency histogram, with load*

```
stdin:2:125-151: WARNING: Return value discarded.
tracepoint:sched:sched_switch /args->next_comm=="lat_pipe"/ { $s=@t[args->next_pid]; if($s){
@off_us=hist((nsecs-$s)/1000); delete(@t[args->next_pid]); } }
```

~~~~~

Attached 2 probes

Pipe latency: 6.4716 microseconds

@off_us:

[2, 4)	311579	@@@@	
[4, 8)	40707	@@@@	
[8, 16)	5454		
[16, 32)	279		
[32, 64)	89		
[64, 128)	59		
[128, 256)	65		
[256, 512)	45		
[512, 1K)	18		
[1K, 2K)	14		
[2K, 4K)	21		
[4K, 8K)	0		
[8K, 16K)	0		
[16K, 32K)	0		
[32K, 64K)	0		
[64K, 128K)	0		
[128K, 256K)	0		
[256K, 512K)	1		
[512K, 1M)	2		

@t[77602]: 1643782344750226

@t[77601]: 1643782344905247

@t[77593]: 1643782345054053

task1/p4_callgraph_noload.txt

perf sched callgraph, no load

```

# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 682K of event 'sched:sched_switch'
# Event count (approx.): 682111
#
# Children      Self  Trace output
# .....
#
16.63%    16.63%  lat_pipe:77440 [120] S ==> swapper/0:0 [120]
|
---0x60deb1a649e5
__libc_start_main_impl (inlined)
call_init (inlined)
__libc_start_call_main
0x60deb1a6496b
0x60deb1a693f4
0x60deb1a68f89
0x60deb1a64d26
0x60deb1a64be8
__libc_read (inlined)
__syscall_cancel
__internal_syscall_cancel (inlined)
entry_SYSCALL_64_after_hwframe
do_syscall_64
x64_sys_call
__x64_sys_read
ksys_read
vfs_read
anon_pipe_read
schedule
__schedule
perf_trace_sched_switch

14.74%    14.74%  lat_pipe:77439 [120] S ==> swapper/3:0 [120]
|
---0x60deb1a649e5
__libc_start_main_impl (inlined)
call_init (inlined)
__libc_start_call_main
0x60deb1a6496b
0x60deb1a693f4
0x60deb1a68fd4
0x60deb1a64b10
__libc_read (inlined)
__syscall_cancel
__internal_syscall_cancel (inlined)
entry_SYSCALL_64_after_hwframe
do_syscall_64
x64_sys_call
__x64_sys_read
ksys_read
vfs_read
anon_pipe_read
schedule
__schedule
perf_trace_sched_switch

12.93%    12.93%  lat_pipe:77442 [120] S ==> swapper/0:0 [120]
|
---0x59949e3f59e5
__libc_start_main_impl (inlined)
call_init (inlined)
__libc_start_call_main

```

```

0x59949e3f596b
0x59949e3fa3f4
|
--12.93%--0x59949e3f9fd4
0x59949e3f5b10
__libc_read (inlined)
__syscall_cancel
__internal_syscall_cancel (inlined)
entry_SYSCALL_64_after_hwframe
do_syscall_64
x64_sys_call
__x64_sys_read
ksys_read
vfs_read

```

Task 2

task2/p10_reboot_confirmed.txt

the reproduced gap + perf stat (dTLB) + smaps (FilePmdMapped)

```

# Task 2 - GAP REPRODUCED after fresh reboot (manual SSH session)
# 22-day uptime had fragmented memory -> huge pages couldn't allocate -> no PMD map -> no gap.
# Fresh boot -> memory defragmented -> v2 folios PMD-mapped -> gap appears. kernel 7.0.0-15-
generic.

=== fresh-boot baseline (verbatim assignment sequence) ===
v1 reads/s: 1,942,670
v2 reads/s: 2,124,442      (+9.4%)

=== perf stat v1 vs v2 (same RUN) ===
--- v1 ---
  reads/s:      1,886,428
  cycles:      13,255,912,155
  instructions: 7,538,218,104
  dTLB-load-misses: 8,790,240
  cache-misses: 130,604,123
  cache-references: 634,898,144
--- v2 ---
  reads/s:      2,082,856
  cycles:      13,263,522,195
  instructions: 8,306,823,225
  dTLB-load-misses: 18,488      <-- ~475x FEWER than v1
  cache-misses: 126,771,643     <-- ~equal to v1 (NOT the driver)
  cache-references: 676,552,167

=== smaps_rollup during a run (huge file mapping?) ===
--- v2 --- reads/s 2,121,280
  Rss:      74,872 kB
  FilePmdMapped: 65,536 kB      <-- whole 64 MiB file PMD (huge-page) mapped
--- v1 --- reads/s 1,938,905
  Rss:      74,972 kB
  FilePmdMapped: 0 kB          <-- v1 gets NO huge mapping

# CONFIRMED CHAIN: prepare 4M writes -> 2MiB PMD-order folios -> PMD huge-page mapping
# (FilePmdMapped=64MiB) -> dTLB-misses collapse 8.79M->18K -> +9-10% reads/s.
# It IS dTLB (cache-misses equal). CONFIG_READ_ONLY_THP_FOR_FS was a red herring; the path
# works here on fresh memory. Earlier non-reproduction = memory fragmentation (22d uptime).

```

task2/p5_folio_hist_v1.txt

folio-order histogram, v1

```
# 5a folio-order histogram - v1 (4K prepare)
# NOTE: kprobe:filemap_alloc_folio NOT probeable on this kernel (inlined).
#       Documented fallback: tracepoint:filemap:mm_filemap_add_to_page_cache, field 'order'
#       (verified via -lv).
#       Histogram bucket = folio order (0=4K page,1=8K,2=16K,...,9=2M PMD). File removed pre-
#       prepare so writes run.
# kernel: 7.0.0-15-generic   Sun Jun 14 11:42:00 UTC 2026
```

```
Attached 1 probe
sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)
```

```
1 files, 65536Kb each, 64Mb total
Creating files for the test...
Extra file open flags: (none)
Creating file test_file.0
67108864 bytes written in 0.05 seconds (1254.36 MiB/sec).
```

```
@order_v1:
[0]          3027 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[1]           0 |
[2]          4096 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
```

task2/p5_folio_hist_v2.txt

folio-order histogram, v2

```
# 5a folio-order histogram - v2 (4M prepare)
# Same probe substitution as v1: tracepoint:filemap:mm_filemap_add_to_page_cache, field 'order'.
#       Histogram bucket = folio order (0=4K,1=8K,2=16K,...,9=2M PMD). File removed pre-prepare
#       so writes run.
# kernel: 7.0.0-15-generic   Sun Jun 14 11:42:10 UTC 2026
```

```
Attached 1 probe
sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)
```

```
1 files, 65536Kb each, 64Mb total
Creating files for the test...
Extra file open flags: (none)
Creating file test_file.0
67108864 bytes written in 0.04 seconds (1467.75 MiB/sec).
```

```
@order_v2:
[0]          2870 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[1]           0 |
[2]           0 |
[3]           0 |
[4]           0 |
[5]           0 |
[6]           0 |
[7]           0 |
[8]           0 |
[9]           32 |
```

task2/p1b_filefrag.txt

filefrag (fragmentation is trivial)

```
/home/ubuntu/v1/test_file.0: 2 extents found
/home/ubuntu/v2/test_file.0: 1 extent found
```

task2/p2_perfstst_v1.txt

perf stat v1 (page-faults are all minor -> fully cached)

sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:

Number of threads: 1

Initializing random number generator from current time

Extra file open flags: (none)

1 files, 64MiB each

64MiB total file size

Block size 4KiB

Number of IO requests: 0

Read/Write ratio for combined random IO test: 1.50

Periodic FSYNC enabled, calling fsync() each 100 requests.

Calling fsync() at the end of test, Enabled.

Using fast mmaped I/O mode

Doing random read test

Initializing worker threads...

Threads started!

File operations:

reads/s: 1882830.28

writes/s: 0.00

fsyncs/s: 0.00

Throughput:

read, MiB/s: 7354.81

written, MiB/s: 0.00

General statistics:

total time: 5.0002s

total number of events: 9417768

Latency (ms):

min: 0.00

avg: 0.00

max: 0.93

95th percentile: 0.00

sum: 3221.33

Threads fairness:

events (avg/stddev): 9417768.0000/0.00

execution time (avg/stddev): 3.2213/0.00

Performance counter stats for 'sysbench fileio --file-num=1 --file-total-size=64M --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run':

1813	page-faults
1813	minor-faults
8744015	dTLB-load-misses
13255640893	cycles
7529345671	instructions

5.008987211 seconds time elapsed

4.992305000 seconds user

0.008995000 seconds sys